An LLM Agent for TRPG Games

Introduction

Tabletop role-playing games (TRPG) is a kind of game that rely on interactions of players and the game master. Unlike board games or video games, TRPGs are not strictly constrained by fixed rules or predefined outcomes. Instead, players are free to decide how their characters interact with the world, how they confront enemies, and how they respond to other events. These choices directly influence how the story develops, making the gameplay highly dynamic and collaborative.

For example, in Dungeons & Dragons, the official rulebooks mainly describe the background of a fantasy world and define the mechanics used during gameplay. These materials explain the history of the setting, the characteristics of different races and creatures, and the rules governing combat and other actions. They specify attributes such as health points, abilities, attack power, and skill systems that determine how characters and monsters behave within the game. However, the actual story scenarios(campaigns) are not fixed. They are typically designed, shared, and improved by the player community rather than fully defined by the official materials.

TRPGs also commonly use dice to introduce randomness into gameplay. Dice rolls determine outcomes such as the amount of damage dealt in combat or whether a character successfully performs an action outside combat. For instance, a character with higher charisma may have a greater chance of persuading another character during dialogue, but the final outcome is often determined by a dice roll. This system provides both strategic depth and uncertainty: players can plan their actions based on character abilities and tactics, while random outcomes ensure that events remain unpredictable.

Because the narrative is not rigidly predetermined, TRPGs support a high degree of creativity and flexibility. Anyone can design new stories that take place in the game world, and game masters can freely adjust the difficulty of encounters, introduce new characters, or modify the direction of the narrative during play. This combination of structured rules and open-ended storytelling is what makes TRPGs fundamentally different from many other types of games.

Popular systems such as Dungeons & Dragons and Call of Cthulhu require the game master to perform complex tasks including deciding combat events (e.g. how enemies move, whether a spell can be performed or not), role-playing as in-game characters (e.g. act like a shop keeper, or anyone that is not acted by a player) and answer questions of players (e.g. history of world and lores). These tasks require in-depth understanding of the game itself and good performance. More importantly, game masters should not only follow rules or game scripts, but also "build" an unforgetting experience with players. The game should be challenging but also allow "miracles" when players are about to lose the game, react to the wits and personalities of players, and in the end, create legends to be told.

Recent advances in large language models have demonstrated strong capabilities in natural language understanding, narrative generation, and reasoning. This creates an opportunity to design an LLM-based agent that can act as a game master. In fact, many have already designed such agents. However, most existing ones' implementations are limited to a single game system and rely heavily on prompt

engineering rather than structured reasoning or memory systems.
This project proposes the design of a **generic TRPG Game Master agent** capable of supporting most rule systems. The system separates rule knowledge, tools, narrative reasoning into modular components and introduces a **self-maintained notebook memory system** that allows both the agent and human users to edit and inspect the game state.

Objectives

The primary objectives of this project are:

1. Design a generic LLM agent capable of running multiple TRPG systems rather than being restricted to a specific rule set.
2. Implement a modular architecture separating rule and world knowledge, and game state management. It should also be highly efficient to save token use.
3. Develop a notebook-based memory system that the agent can update, summarize, and query during gameplay. It should be based on natural language, so it allows manual corrections and debugging.
4. Provide tool-based interactions (e.g., dice rolling and notebook querying) to ensure reliable and interpretable decision-making.

Model Overview

The proposed system consists of three primary components:

1. **Narrator Agent (LLM)**
2. **Rule and Lore Retrieval System (RAG)**
3. **Notebook-Based Memory System**

The agent also interacts with the game environment through a limited set of tools such as roll dice (which most TRPGs require, and you won't rely on LLMs to provide "random rolls", will you?).
The LLM acts as the central reasoning engine and is in charge of the interaction among the players, the game state, and external tools. The LLM interprets player actions, consults rules or world knowledge when necessary, updates the notebook memory, and generates narrative responses.
To support multiple games, rules are stored externally and accessed using RAG. The system maintains separate vector databases for game rules and world lore. When a player action depends on a rule interpretation, the agent retrieves relevant rule passages and uses them to guide decision-making.
A key innovation of this project is a persistent notebook memory that stores the evolving state of the game. The notebook is implemented as a structured collection of human-readable documents (e.g., Markdown or JSON files). It contains multiple sections such as player status, NPC profiles, Combat logs and Summaries.
The notebook system provides several advantages:

1. Persistent memory across gameplay sessions.
2. Human editability for debugging and correction.
3. Structured querying by the agent.
4. Automatic summarization to prevent memory growth.
5. Generic games/rules support by user-defined sections.

The agent periodically summarizes long logs into concise summaries that can be retrieved efficiently during reasoning.

Example

The agent operates in a loop during gameplay:

1. Receive player input.
2. Interpret the player's intended action.
3. Retrieve rules or world knowledge if necessary.
4. Query notebook state when relevant.
5. Perform dice rolls if required.
6. Update the notebook with new events.
7. Generate narrative output.

This loop enables the agent to simulate the responsibilities of a human game master.

Example:

1. Player declares an action. (Wizard casts “fireball”)
2. Agent checks whether rule clarification is required. (“Fireball” is a spell, should check its entries for damage and range.)
3. Relevant rule passages are retrieved. (“Fireball” requires within 150 feet range. A target takes 8d6 fire damage on a failed Dexterity save, or half as much damage on a successful one.) (This one is “translated” from vector to natural language, for better understanding.)
4. The agent determines whether the action is valid. (Read notebook to find that the enemy is within 150 feet where the player can see.)
5. Dice rolls and outcomes are generated (Enemy takes 24 damage and is updated on notebook).

This design allows the system to support multiple rule systems without retraining the model.

Dataset and Measurement

We will be using DnD5e official rule book and additional lore book as dataset. Depending on its effeteness, we might also use the community wiki of DnD5e and perhaps some exemplar games held by real humans.

The measurement can be vague, as it is hard to tell what a good game master can be. I would give the following:

1. Utilize the notebook and tools at the right time.
2. Correctly interpret actions of players. Correctly determine whether an action is able to be performed.
3. Act as NPCs without being out of character. Act as enemies to be reasonably strong (at least not dumb).
4. Communicate with players as a game master, such as giving them hints, or jumping in conversations.
5. Finish a short campaign of DnD5e, without being noticeably “buggy”.

Conclusion

This project explores how LLM agents can act as generalized game masters for tabletop role-playing games. By combining retrieval-based rule interpretation, tool-based reasoning, and a persistent notebook memory system, the proposed architecture aims to provide a flexible and extensible framework for automated TRPG gameplay.

This project aims to contribute the following:

1. A generic architecture for TRPG game-master agents that supports multiple rule systems.
2. A notebook-based memory framework that enables both agent-driven and human-editable state management.

Appendix

This project is currently a “group” project by 宋益善(3036649034) only.